CMPS 350 Project – Report

Education Platform

https://github.com/mdsherinoff/cmps350-project

1. Description of the proposed platform

A login system comprising of a negligibly minimalistic design of a gradient color theme attempts to take in user information, divided into a student, an instructor, and a departmental admin. Each login takes the user to their respective pages, where the student can view and register for courses with specific sections enabled, while at the same time note down his/her learning path to be followed to complete his degree. It also displays the varying grades and courses in progress.

An instructor can picture the prospective courses he must undertake and envision the grading mechanisms to be assigned to the students for the courses he teaches. The admin is given the authorization to manage classes and validate courses to be open for registration. He also has the authority to create newer classes and sections. While at the same time view the schedule of courses in progress.

An extensive display of immersive and dedicated statistics is collected remotely through server actions.

2. Data Model

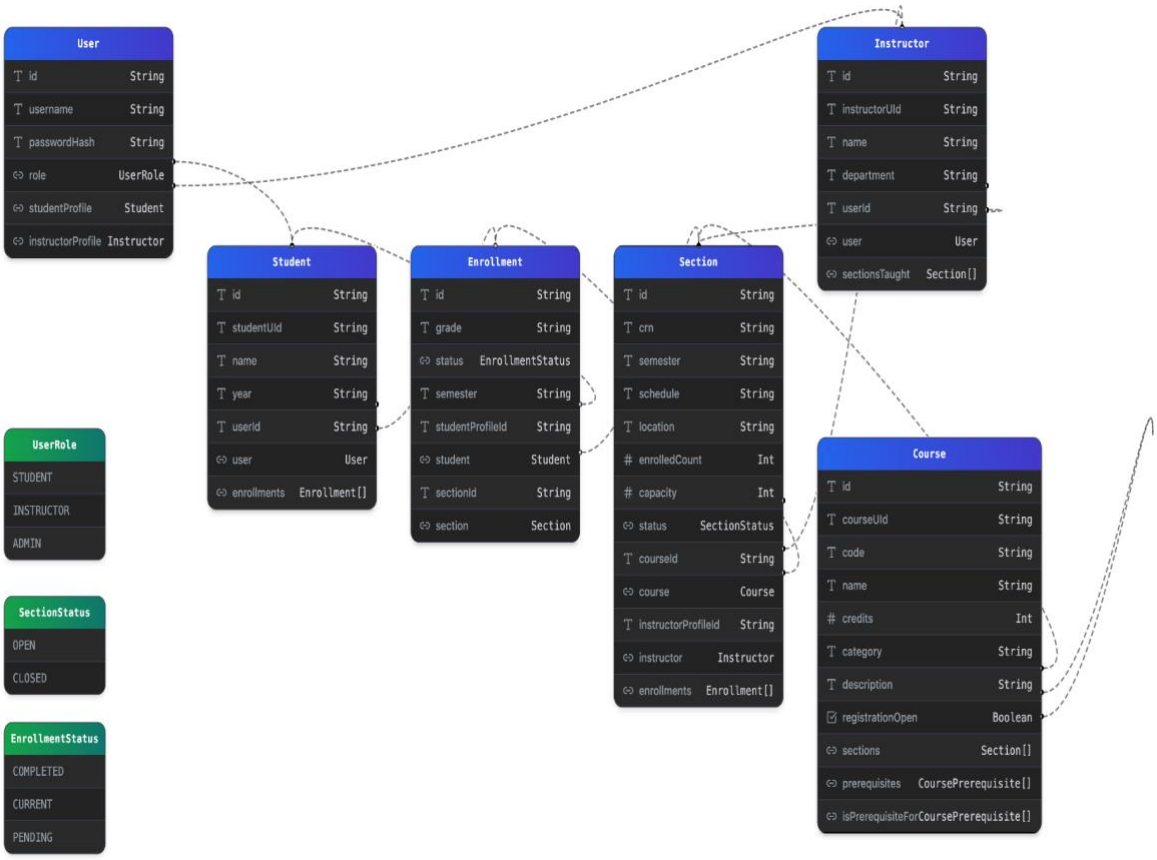

Figure 1: The Entity Diagram

Following the Entity diagram, the Prisma schema showcase concrete analysis and conversion into proper documentation:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}

enum UserRole {
  STUDENT
  INSTRUCTOR
  ADMIN
}

enum SectionStatus {
  OPEN
  CLOSED
}

enum EnrollmentStatus {
  COMPLETED
  CURRENT
  PENDING
}


model User {
  id           String   @id @default(uuid())
  username     String   @unique
  passwordHash String
  role         UserRole

  // Relations
  studentProfile    Student?
  instructorProfile Instructor?

  @@map("users")
}


model Student {
  id         String @id @default(uuid())
  studentUId String @unique
  name       String
  year       String


  // Relation to User (one-to-one)
  userId String @unique
  user   User   @relation(fields: [userId], references: [id], onDelete: Cascade)


  // Relation to Enrollments (one-to-many)
  enrollments Enrollment[]
```

```
  @@map("student_profiles")
}
model Instructor {
  id            String   @id @default(uuid())
  instructorUId String   @unique
  name          String
  department    String


  // Relation to User (one-to-one)
  userId String @unique
  user   User   @relation(fields: [userId], references: [id], onDelete: Cascade)


  // Relation to Sections (one-to-many)
  sectionsTaught Section[]

  @@map("instructor_profiles")
}


model Course {
  id        String @id @default(uuid())
  courseUId String @unique
  code      String @unique
  name      String
  credits   Int
  category  String

  description      String?
  registrationOpen Boolean  @default(false)

  // Relation to Sections (one-to-many)
  sections Section[]

  prerequisites     CoursePrerequisite[] @relation("CourseOffering")
  // Lists courses for which *this course is a prerequisite*
  isPrerequisiteFor CoursePrerequisite[] @relation("PrerequisiteCourse")

  @@map("courses")
}


// Join table for Course prerequisites (many-to-many self-relationship on Course)
model CoursePrerequisite {
  // Foreign Keys forming a composite Primary Key
  courseId             String
  prerequisiteCourseId String


  // Relations
  course       Course @relation("CourseOffering", fields: [courseId], references: [id],
onDelete: Cascade)
  prerequisite Course @relation("PrerequisiteCourse", fields: [prerequisiteCourseId],
references: [id], onDelete: Cascade)

  @@id([courseId, prerequisiteCourseId])
  @@map("course_prerequisites")
}
```

```
model Section {
  id            String        @id @default(uuid())
  crn           String        @unique
  semester      String
  schedule      String
  location      String
  enrolledCount Int           @default(0)
  capacity      Int
  status        SectionStatus @default(CLOSED)


  // Relation to Course (many-to-one)
  courseId String
  course   Course @relation(fields: [courseId], references: [id], onDelete: Cascade) // If
Course is deleted, delete its Sections


  // Relation to Instructor (many-to-one)
  // Making instructorProfileId optional allows a section to exist temporarily without an
assigned instructor
  instructorProfileId String?
  instructor          Instructor? @relation(fields: [instructorProfileId], references: [id],
onDelete: SetNull)


  // Relation to Enrollments (one-to-many)
  enrollments Enrollment[]

  @@index([courseId])
  @@index([instructorProfileId])
  @@map("sections")
}


model Enrollment {
  id       String           @id @default(uuid())
  grade    String?
  status   EnrollmentStatus
  semester String


  // Relation to Student(many-to-one)
  studentProfileId String
  student          Student @relation(fields: [studentProfileId], references: [id], onDelete:
Cascade)


  // Relation to Section (many-to-one)
  sectionId String
  section   Section @relation(fields: [sectionId], references: [id], onDelete: Cascade)

  @@unique([studentProfileId, sectionId])
  @@index([studentProfileId])
  @@index([sectionId])
  @@map("enrollments")
}
```

3. Web API, Server Actions and repository

The repository includes a ***courses-repo.js***, an ***instructors-repo.js,*** a ***students-repo.js***, a ***users-repo.js*** and a ***masters-repo.js***.

Data is queried using the following server actions:

i) findUserByUsernameAction(username)

This method calls an export method from ***master-repo.js*** called findUserByUsername(username), which takes username as an input and searches it in the database using prisma, and returns wherever true.

```
async findUserByUsername(username) {
    return await prisma.user.findUnique({
      where: { username },
      include: {
        studentProfile: true,
      },
    });
  }
```

ii) findStudentProfileByUIdAction(studentUId)

This method calls an export method from ***master-repo.js*** called findStudentProfileByUI(studentUId), which takes a student's id as an input and searches it in the database using prisma, and returns the student's profile wherever true.

```
async findStudentProfileByUId(studentUId) {
    return await prisma.StudentProfile.findUnique({
      where: { studentUId },
      include: {
        user: true,
        enrollments: {
          orderBy: { semester: "desc" },
          include: {
            section: {
              include: {
                course: { select: { code: true, name: true,
courseUId: true } },
                instructor: { select: { name: true,
instructorUId: true } },
              },
            },
```

```
        },
      },
    },
  });
}
```

iii) findInstructorProfileByUIdAction(instructorUId)

This method calls an export method from ***master-repo.js*** called findInstructorProfileByUId(instructorUId) which takes an instructors's id as an input and searches it in the database using prisma, and returns the instructor's profile wherever true.

```
async findInstructorProfileByUId(instructorUId) {
    return await prisma.InstructorProfile.findUnique({
      where: { instructorUId },
      include: {
        user: { select: { username: true, role: true } },
        sectionsTaught: {
          orderBy: { semester: "desc" },
          include: {
            course: { select: { code: true, name: true,
courseUId: true } },
          },
        },
      },
    });
  }
```

iv) getAllStudentsAction()

This method calls an export method from ***master-repo.js*** called getAllStudents() which searches the database using prisma, and returns every student.

```
async getAllStudents() {
    return await prisma.student.findMany({
      include: {
        user: { select: { username: true } },
      },
    });
  }
```

v) findCourseByUIdAction(courseUId)

This method calls an export method from ***master-repo.js*** called findCourseByUId(courseUId) which takes a course's id as an input and searches it in the database using prisma, and returns the Course wherever true.

```
async findCourseByUId(courseUId) {
    return await prisma.course.findUnique({
      where: { courseUId },
      include: {
        sections: {
          orderBy: { crn: "asc" },
          include: {
            instructor: {
              select: { name: true, instructorUId: true,
department: true },
            },
          },
        },
        prerequisites: {
          select: {
            prerequisite: {
              select: { code: true, name: true, courseUId:
true },
            },
          },
        },
        isPrerequisiteFor: {
          select: {
            course: { select: { code: true, name: true,
courseUId: true } },
          },
        },
      },
    });
  }
```

vi) findCourseByCodeAction(courseCode)

This method calls an export method from ***master-repo.js*** called findCourseByCode(courseCode) which takes a course's code as an input and searches it in the database using prisma, and returns the course wherever true.

```
async findCourseByCode(courseCode) {
    return await prisma.course.findUnique({
      where: { code: courseCode },
      include: {
        sections: {
          orderBy: { crn: "asc" },
          include: {
            instructor: {
              select: { name: true, instructorUId: true,
department: true },
            },
          },
        },
        prerequisites: {
          select: {
            prerequisite: {
              select: { code: true, name: true, courseUId:
true },
            },
          },
        },
        isPrerequisiteFor: {
          select: {
            course: { select: { code: true, name: true,
courseUId: true } },
          },
        },
      },
    });
  }
```

vii) getEnrollmentsForStudentAction(studentUId)

This method calls an export method from ***master-repo.js*** called getEnrollmentsForStudent(studentUId) which takes a student's id as an input and searches it in the database using prisma, and returns the courses enrolled wherever true.

```
async getEnrollmentsForStudent(studentUId) {
    const studentProfile = await
prisma.studentProfile.findUnique({
      where: { studentUId },
      select: { id: true },
    });
```

```
    if (!studentProfile) {
      console.warn(`Student with UId "${studentUId}" not
found.`);
      return [];
    }

    return await prisma.enrollment.findMany({
      where: { studentProfileId: studentProfile.id },
      orderBy: [{ semester: "desc" }, { section: { course: {
code: "asc" } } }],
      include: {
        section: {
          include: {
            course: { select: { code: true, name: true,
credits: true } },
            instructor: { select: { name: true } },
          },
        },
      },
    });
  }
```

viii) getStudentsEnrolledInSectionAction(sectionCRN)

This method calls an export method from ***master-repo.js*** called getStudentsEnrolledInSection (sectionCRN) which takes a course's CRN as an input and searches it in the database using prisma, and returns the students enrolled wherever true.

```
async getStudentsEnrolledInSection(sectionCRN, skip = 0, take
= 20) {
    const section = await prisma.section.findUnique({
      where: { crn: sectionCRN },
      select: { id: true },
    });

    if (!section) {
      console.warn(`Section with CRN "${sectionCRN}" not
found.`);
      return [];
    }

    return await prisma.enrollment.findMany({
```

```
      where: { sectionId: section.id },
      skip,
      take,
      orderBy: { student: { name: "asc" } },
      include: {
        student: {
          include: {
            user: { select: { username: true } },
          },
        },
      },
    });
}
```

ix) createCourseAction(courseData)

This method calls an export method from ***master-repo.js*** called createCourseAction(courseData) which takes a course's data as an input and creates a course, and stores it in the database using prisma.

```
async createCourse(courseData) {
    const newCourse = await prisma.course.create({
      data: {
        courseUId: courseData.id,
        code: courseData.code,
        name: courseData.name,
        credits: courseData.credits,
        category: courseData.category,
        description: courseData.description || null,
        sections: course.sections,
        prerequisites: courseData.prerequisite,
      },
    });
    return newCourse;
  }
```

x) updateStudentGradeAction(courseData)

This method calls an export method from ***master-repo.js*** called updateStudentGradeAction(courseData) which takes a course's data as an input and searches the database using prisma, and updates it using prisma.

```
async updateStudentGrade(studentUId, crn, grade) {
    const student = await prisma.student.findUnique({
      where: { studentUId },
      select: { studentUId: true },
    });

    if (!student) {
      throw new Error(`Student with UId "${studentUId}" not
found.`);
    }

    const section = await prisma.section.findUnique({
      where: { crn },
      select: { id: true },
    });

    if (!section) {
      throw new Error(`Section with CRN "${crn}" not found.`);
    }

    const updatedEnrollment = await prisma.enrollment.update({
      where: {
        studentProfileId: student.studentUId,
        sectionId: section.id,
      },
      data: {
        grade,
      },
    });
```

xi) registerStudentInCourseAction(courseData)

This method calls an export method from ***master-repo.js*** called registerStudentsInCourse(courseData) which takes a course's data as an input and allows the student to be added to the course in database using prisma.

```
async registerStudentInCourse(studentUId, courseUId, crn) {
    const studentProfile = await
prisma.StudentProfile.findUnique({
      where: { studentUId },
      select: { id: true },
    });
```

```
    if (!studentProfile) {
      throw new Error(`Student with UId "${studentUId}" not
found.`);
    }

    const section = await prisma.enrollment.findUnique({
      where: { crn },
      select: { id: true },
    });

    if (!section) {
      throw new Error(`Section with CRN "${crn}" not found.`);
    }

    const newEnrollment = await prisma.enrollment.create({
      data: {
        studentProfileId: studentProfile.id,
        sectionId: section.id,
      },
    });

    return newEnrollment;
  }
```

xii) getInstructorAnalyticsAction()

This method calls an export method from ***master-repo.js*** called getInstructorAnalytics() which returns all Instructor statistics from the database using prisma.

```
async getInstructorAnalytics() {
    // Total number of instructors
    const totalInstructors = await prisma.instructor.count();

    // Instructors by department
    const instructorsByDepartment = await
prisma.instructor.groupBy({
      by: ["department"],
      _count: {
        instructorUId: true,
      },
    });
```

```
    const allInstructorsWithSections = await
prisma.instructor.findMany({
      include: {
        sectionsTaught: true,
      },
    });

    // Instructors with no sections
    const instructorsWithNoSections =
allInstructorsWithSections.filter(
      (instructor) => instructor.sectionsTaught.length === 0
    ).length;

    // Instructors with most sections
    const instructorsWithSections = allInstructorsWithSections
      .map((instructor) => ({
        name: instructor.name,
        department: instructor.department,
        sectionsCount: instructor.sectionsTaught.length,
      }))
      .sort((a, b) => b.sectionsCount - a.sectionsCount);

    const maxSections =
instructorsWithSections[0]?.sectionsCount || 0;

    const instructorsWithMostSections =
instructorsWithSections.filter(
      (instructor) => instructor.sectionsCount === maxSections
    );

    return {
      totalInstructors,
      instructorsByDepartment,
      instructorsWithMostSections,
      instructorsWithNoSections,
    };
  }
```

xiii) getCourseAnalyticsAction()

This method calls an export method from ***master-repo.js*** called getCourseAnalytics() which returns all course statistics from the database using prisma.

```
async getCourseAnalytics() {
    // Total number of courses
    const totalCourses = await prisma.course.count();

    // Courses by category
    const coursesByCategory = await prisma.course.groupBy({
      by: ["category"],
      _count: {
        courseUId: true,
      },
    });

    const allCourses = await prisma.course.findMany({
      include: {
        sections: {
          include: {
            enrollments: {
              where: {
                grade: "A",
              },
            },
          },
        },
      },
    });

    // Total A grade for each course
    const coursesWithACounts = allCourses.map((course) => ({
      ...course,
      aCount: course.sections.reduce(
        (sum, section) => sum + section.enrollments.length,
        0
      ),
    }));

    // Courses with the maximum number of A grades
    const maxACount = Math.max(
      ...coursesWithACounts.map((course) => course.aCount)
    );

    // Courses with most A grades
    const coursesWithMostAs = coursesWithACounts
      .filter((course) => course.aCount === maxACount)
      .sort((a, b) => a.code.localeCompare(b.code));
```

```
    // Courses open for registration
    const coursesWithOpenRegistration = await
prisma.course.count({
      where: {
        registrationOpen: true,
      },
    });

    return {
      totalCourses,
      coursesByCategory,
      coursesWithMostAs,
      coursesWithOpenRegistration,
    };
  }
```

xiv) getStudentAnalyticsAction(courseData)

This method calls an export method from ***master-repo.js*** called getStudentAnalytics() which returns all student statistics from the database using prisma.

```
async getStudentAnalytics() {
    // Total number of students
    const totalStudents = await prisma.student.count();

    const studentsWithGrades = await prisma.student.findMany({
      include: {
        enrollments: {
          include: {
            section: {
              include: {
                course: true,
              },
            },
          },
        },
      },
    });

    // GPA for each student
    const studentsWithGPA = studentsWithGrades.map((student)
=> {
```

```
      const grades = student.enrollments
        .filter((e) => e.grade)
        .map((e) => {
          const gradeMap = {
            A: 4.0,
            "A-": 3.7,
            "B+": 3.3,
            B: 3.0,
            "B-": 2.7,
            "C+": 2.3,
            C: 2.0,
            "C-": 1.7,
            "D+": 1.3,
            D: 1.0,
            "D-": 0.7,
            F: 0.0,
          };
          return {
            grade: gradeMap[e.grade] || 0,
            credits: e.section.course.credits,
          };
        });

      const totalCredits = grades.reduce((sum, g) => sum +
g.credits, 0);
      const gpa =
        totalCredits > 0
          ? grades.reduce((sum, g) => sum + g.grade *
g.credits, 0) /
            totalCredits
          : 0;

      return {
        ...student,
        gpa: Number(gpa.toFixed(2)),
      };
    });

    // Top 5 students by GPA
    const topStudentsByGPA = studentsWithGPA
      .sort((a, b) => b.gpa - a.gpa)
      .slice(0, 5);

    // Students in each year
    const studentsByYear = await prisma.student.groupBy({
```

```
      by: ["year"],
      _count: {
        studentUId: true,
      },
    });

    return {
      totalStudents,
      topStudentsByGPA,
      studentsByYear,
    };
  }
}
```

4. Implemented statistics use case

4.1. User Interface

Once the Admin logs in into the app, he is able to view Course Statistics on the Course Statistics tab on the navigation bar. The stats can be classified under Students, Courses and Instructors.

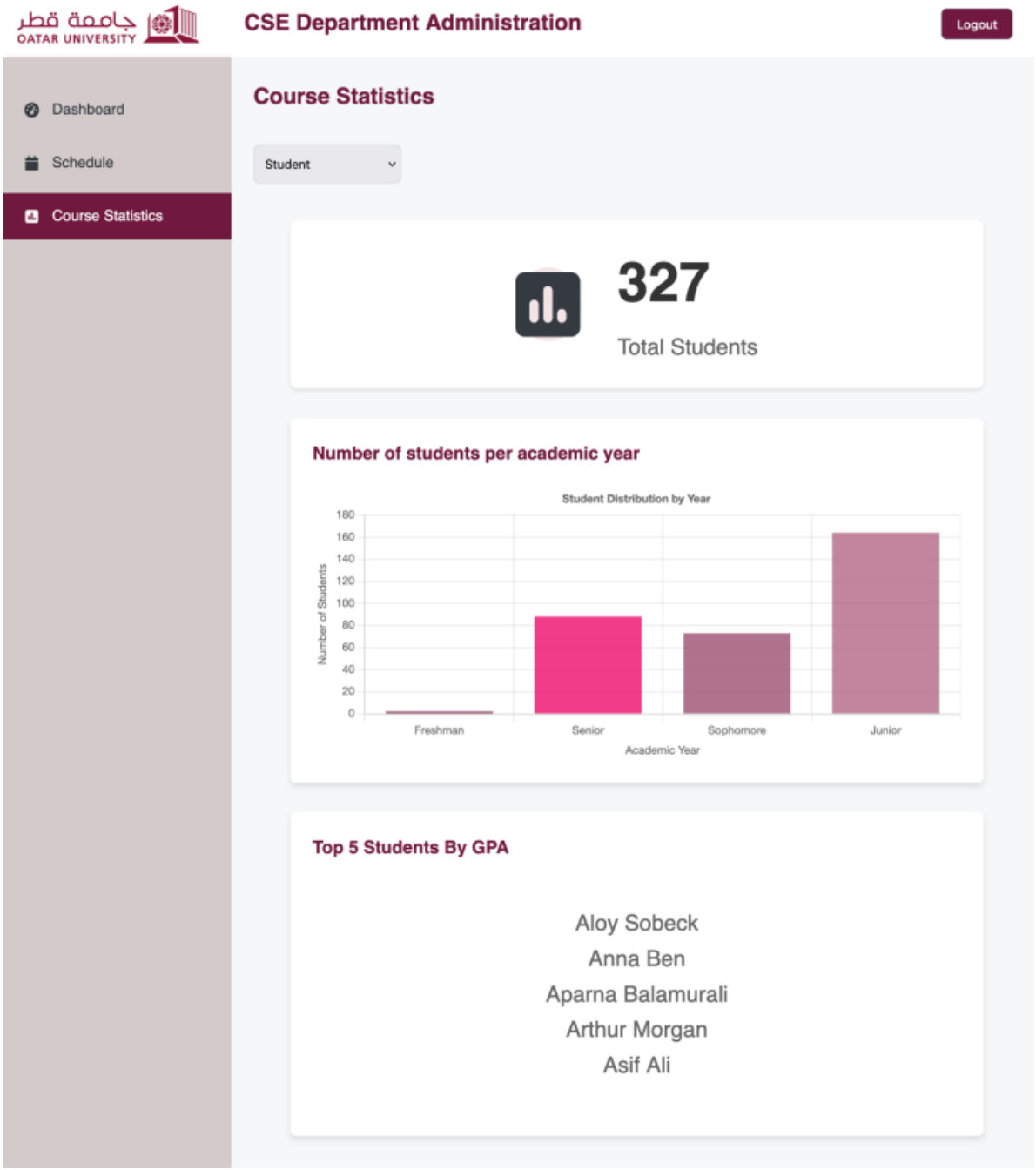

Figure 2: Student's Statistics

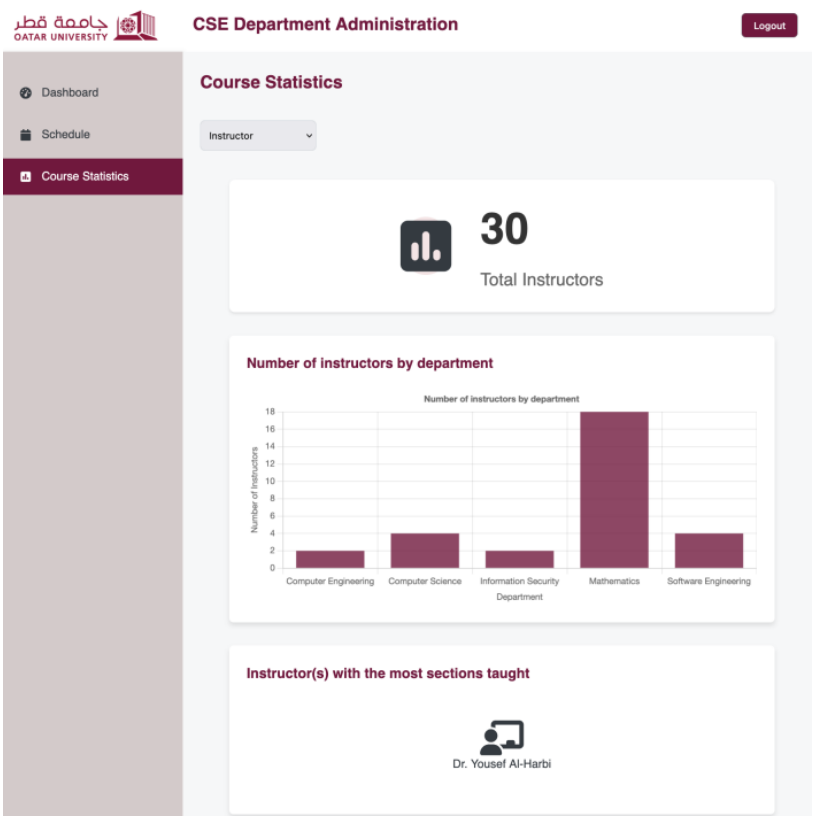

Figure 3: Instructors Statistics

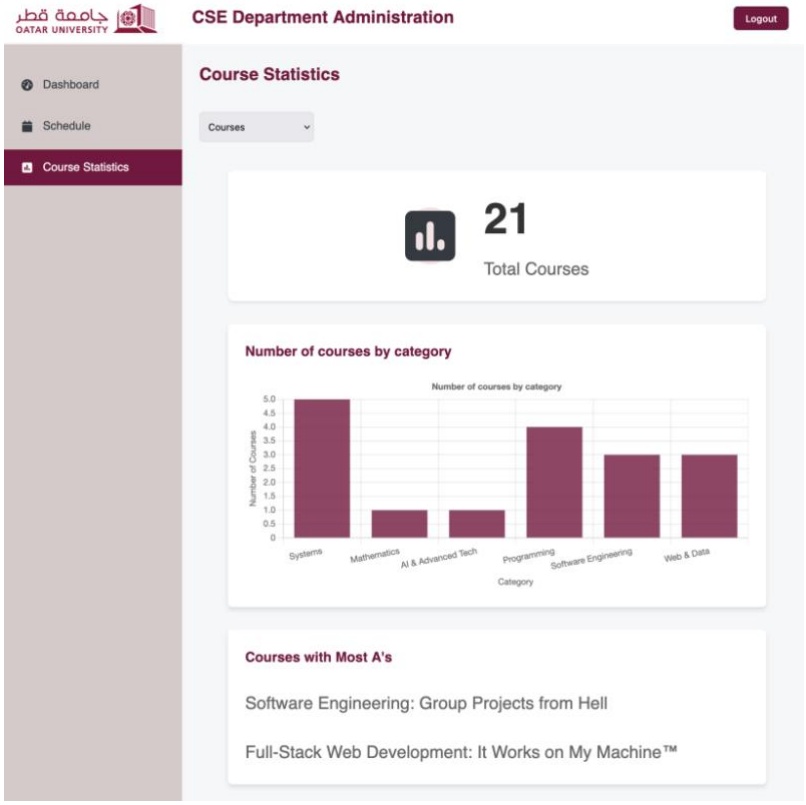

Figure 4: Courses Statistics

4.2. Implemented queries

Refer to (3) Web API, Server Actions, and Repository.

4.3. Data used in the statics

The statistics are based on comparisons and methods implemented within the database tables using Prisma. **Refer to (3) Web API, Server Actions, and Repository.**

5. Implemented Computer Authentication use case

Authentication is done through google and GitHub and using JWT on the server end.

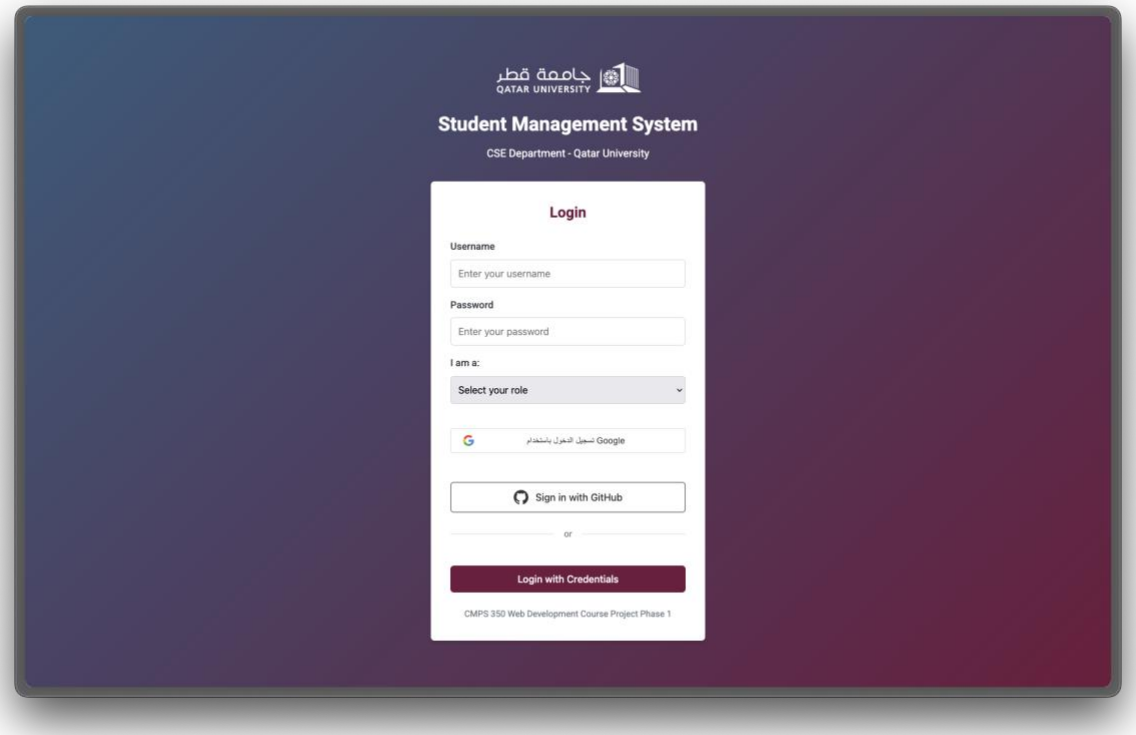

Figure 5: Login Screen with Authentication

6. Discussion of the project contribution of each team member

Student name	Student contributions
Aman	26
Mishal	26
Sherin	26
Ameen	22

Aman and Mishal were responsible for the conversion of the project from the phase one Json files into database and inculcated Prisma into it. Sherin provided a more concrete understanding of the cases while seeding the database and completing the server actions leading to the statistics page, with complete support from Aman, and Ameen. Ameen was responsible for the troubleshooting of server actions and the authentication factor, collaborating with Mishal and his extensive knowledge of authentication in the process.